Pydantic과 FastAPI를 활용한 데이터 검증과 API 설계

Day6 수업 코드의 개념별 분석

수업 정리 논문 | 2026년 10월 21일

초록

본고는 Day6의 Python 수업 코드 14개를 분석하여 데이터 모델링, 선언적 제약, 사용자 정의 검증, 중첩 구조, 직렬화, 부분 수정, HTTP API 구현의 관계를 정리한다. 핵심 주장은 입력을 모델로 검증하는 절차와 수정 후 상태를 다시 검증하는 절차를 함께 설계해야 데이터 규칙을 일관되게 유지할 수 있다는 것이다. 코드 정적 분석, 개별 실행, FastAPI TestClient 요청을 통해 이를 확인하였다. BaseModel은 타입 변환과 필수 입력 검사를 수행하며, Field는 값의 범위를 제한한다. 검증기는 전처리와 필드 간 관계를 다루고, 중첩 모델은 복합 데이터의 내부 오류 위치를 드러낸다. 직렬화에서는 값의 표현과 필드의 입력 여부를 구분해야 한다. CRUD 예제에서는 변경 필드만 병합한 뒤 최종 모델로 재검증하여 잘못된 null 입력을 거부하였다. 실습 파일의 변수 오류와 미완성 구현도 확인하였으며, 이를 개념 설명과 구분하여 기술한다.

주요어 Pydantic · BaseModel · 데이터 검증 · 직렬화 · PATCH · FastAPI

1 서론

외부에서 받은 데이터는 프로그램이 기대하는 구조와 다를 수 있다. 나이와 가격이 문자열로 전달되거나, 필수 항목이 빠지거나, 개별 값은 올바르지만 서로의 관계가 잘못될 수 있다. 일반 Python 클래스의 타입 힌트만으로는 이러한 입력을 실행 시점에 자동으로 차단하지 않는다. 따라서 데이터가 애플리케이션에 들어오는 지점에서 타입과 업무 규칙을 명시적으로 검사할 필요가 있다.

Day6 수업은 이러한 문제를 모델 선언에서 시작하여 웹 API의 요청과 응답까지 확장한다. 본고는 파일 순서를 그대로 나열하기보다 각 개념의 역할, 처리 순서, 다른 개념과의 연결을 설명한다. 읽는 순서는 모델의 구조, 값의 제약, 검증 시점, 중첩 데이터, 외부 표현, 수정 상태, API 적용 순이다. 실험 결과는 해당 수업 예제의 기능 확인을 뜻하며, 처리 성능이나 운영 환경의 신뢰성을 측정한 결과는 아니다.

2 분석 자료와 방법

분석 대상은 C:/AIO4/Day6/06_Pydantic_데이터검증_FastAPI 아래의 Python 소스 14개이다. __pycache__의 바이트코드 파일은 소스 분석 대상에서 제외하였다. 수업 자료를 1차 근거로 사용하고, Pydantic과 FastAPI의 공식 문서는 API 의미를 확인하는 보조 근거로 사용하였다. 문서 전체에서 [L1]부터 [L6]은 수업 소스 묶음, [R1]부터 [R7]은 공식 문서를 가리킨다.

2.1 개념과 소스의 대응

구분	파일 위치와 이름	주요 개념
[L1] 5개	ch.02_basemodel/ day6_1.py, day6_2.py, day6_2_1.py, day6_2_1_ex.py, main.py	일반 클래스와 모델 비교 타입, 기본값, 상속, import
[L2] 2개	ch.03_Field/ demo.py, exercise.py	범위, 길이, 정규식 EmailStr, Literal, strict
[L3] 1개	ch.04_validator/demo.py	필드 전처리와 정규화 비밀번호와 날짜 관계
[L4] 2개	ch.05_nested/ demo.py, exercise.py	중첩 모델, Enum 합계와 평균, 오류 경로
[L5] 2개	ch.06_serialization/ demo.py, exercise.py	딕셔너리와 JSON 입력 여부와 부분 수정
[L6] 2개	ch07_FastAPI/ app.py, exercise_app.py	요청과 응답 모델 메모리 CRUD와 HTTP 오류

2.2 실행 환경과 검증 절차

실행 환경은 Windows의 프로젝트 가상환경이며 Python 3.12.14, Pydantic 2.14.1, FastAPI 0.150.0을 확인하였다. 각 Python 파일을 해당 파일의 폴더를 작업 디렉터리로 하여 별도 프로세스에서 실행하였다. 이어 검증 순서와 경계값을 확인하는 추가 입력을 구성하고, app.py는 TestClient를 통해 13개 요청을 순서대로 수행하였다. API 실습 파일에는 생성 요청을 별도로 보내 미구현 응답을 확인하였다.

정상 종료 여부와 기능 완성 여부는 별도로 판단하였다. 모델 정의만 있는 파일이나 서버 객체만 만드는 파일은 실행이 정상 종료되어도 모든 기능이 검증된 것은 아니다. 실험 중 원본 수업 소스는 수정하지 않았다. 분석 결과와 수정 제안은 문서에만 기록하였다.

3 BaseModel과 타입 기반 데이터 모델링

3.1 일반 클래스와 검증 모델

day6_1.py의 NormalClass는 생성자에서 받은 값을 속성에 대입한다. name: str, age: int라는 표시는 개발자와 도구에 타입 정보를 제공하지만, 이 클래스 자체에는 잘못된 입력을 거부하는 코드가 없다. 반면 BaseModel을 상속한 모델은 선언된 필드를 읽어 객체 생성 시 검증한다. 이 차이가 수업 전체의 출발점이다. [L1]

```
from pydantic import BaseModel

class Member(BaseModel):
    username: str
    age: int
    active: bool = True
    nickname: str | None = None

member = Member.model_validate({"username": "kim", "age": "34"})
# member.age == 34; member.active is True
```

위 예시는 수업의 Member를 발췌한 것이다. model_validate는 딕셔너리를 검증해 모델 객체를 만든다. 생성자 Member(...)도 검증을 수행한다. 기본 설정에서는 허용되는 변환이 적용되므로 정수 필드의 문자열 "34"가 정수 34가 된다. 숫자로 해석할 수 없는 "스물셋"까지 변환되는 것은 아니다. 검증의 목표는 입력을 그대로 보존하는 것보다 선언된 규칙을 만족하는 결과를 만드는 데 있다. [L1; R1]

3.2 필수 입력과 None 허용

필드 선언	생략	None 입력	의미
name: str	불가	불가	문자열을 반드시 전달
note: str | None	불가	가능	값은 nullable이지만 필수
note: str | None = None	가능	가능	생략 시 None을 사용
active: bool = True	가능	불가	생략 시 True를 사용

Optional[str]은 str | None, Union[int, str]은 int | str와 같은 종류의 타입 표현이다. Optional이라는 이름만으로 생략 가능하다고 해석하면 안 된다. Pydantic v2에서 생략 가능 여부는 기본값 유무와 함께 판단해야 한다. Student의 student_id는 정수 또는 문자열을 허용하므로 문자열 학번을 반드시 정수로 바꾼다고 단정할 수도 없다. [L1; R1]

3.3 모델 상속과 실행 진입점

MemberOut(Member)는 기존 필드를 물려받고 id를 추가한다. 이 방식은 뒤의 TaskIn과 TaskOut 분리에 연결된다. 또한 if __name__ == "__main__" 조건은 파일을 직접 실행할 때만 main()을 호출하게 한다. import 자체가 아무 코드도 실행하지 않는다는 뜻은 아니다. 실제로 조건문 바깥의 문장은 import 시에도 실행될 수 있다. [L1; L4]

4 Field를 통한 선언적 제약

4.1 타입 검사와 값의 범위

타입이 맞아도 서비스에서 허용하지 않는 값일 수 있다. 나이가 정수여도 가입 연령 범위를 벗어나거나, 상품명이 문자열이어도 너무 짧을 수 있다. Registration은 Field를 사용하여 이러한 조건을 모델 선언에 포함한다. age의 ge=14, le=100은 14 이상 100 이하를 뜻하며, name의 min_length와 max_length는 문자열의 길이를 제한한다. [L2]

도구	수업 예시	검증 대상
gt / ge	product_id: Field(gt=0) price: Field(ge=0)	초과 / 이상
le	score: Field(le=100)	이하
min_length / max_length	name의 길이 items의 최소 원소 수	문자열 길이와 컨테이너 길이
pattern	^STU-[0-9]{4}$	문자열의 형식
Literal	Literal["basic", "premium"]	허용된 고정값
EmailStr	email: EmailStr	이메일 주소 구문

숫자의 상한을 max_length로 지정하면 의미가 다르다. 점수 0~100은 ge=0, le=100으로 표현한다. 리스트의 min_length=1은 원소가 최소 한 개 있어야 한다는 뜻이며, 원소 내부의 점수 조건은 별도의 중첩 모델이 검사한다. [L2; L4]

4.2 엄격한 타입과 모델 설정

Registration의 agreed와 API의 done에는 Field(strict=True)가 지정되어 있다. API 확인에서 JSON 불리언 true는 허용되지만 문자열 "true"는 422로 거부되었다. 엄격성은 해당 필드에 적용된다. agreed가 엄격하다고 해서 age="50"까지 금지되는 것은 아니며, 실제 수업의 나이 입력은 정수로 변환된다. [L2; L6; R4]

ConfigDict(extra="forbid")는 선언하지 않은 키를 오류로 처리한다. str_strip_whitespace=True는 문자열의 앞뒤 공백을 제거한다. 이 조합을 사용한 TaskIn에서 title=" "는 공백 제거 후 빈 문자열이 되어 최소 길이 조건에 실패했다. 다만 모든 모델에 이 설정이 적용된 것은 아니다. Product에는 별도 extra 설정이 없으므로 오타 키가 검증 실패를 보장하지 않는다. [L2; L6]

4.3 제약의 경계와 해석

EmailStr 검증은 이메일 소유권이나 실제 수신 가능성 확인을 뜻하지 않는다. 정규식도 의도한 범위를 정확히 지정해야 한다. Product의 원본 패턴 SKU-[0-9]{4}$에는 시작 앵커 ^가 없으므로 XXSKU-0001이 통과함을 확인했다. 정확히 SKU-0001 형태만 허용하려면 ^SKU-[0-9]{4}$와 같이 양끝을 제한하는 수정이 필요하다. [L2; R2]

5 사용자 정의 검증과 실행 순서

5.1 전처리와 필드 검증

Field만으로 표현하기 어려운 변환이나 규칙에는 field_validator를 사용한다. Signup의 tags 검증기는 mode="before"로 원시 입력을 먼저 받는다. 문자열 "python, api"를 쉼표로 나누고 공백과 빈 항목을 제거한 뒤 list[str] 검증으로 넘긴다. 이미 리스트가 들어오면 그대로 반환한다. 전처리 함수는 다양한 원시 입력 타입을 받을 수 있음을 고려해야 한다. [L3]

```
@field_validator("tags", mode="before")
@classmethod
def parse_tags(cls, value: object) -> object:
    if isinstance(value, str):
        return [s.strip() for s in value.split(",") if s.strip()]
    return value
```

mode를 생략한 필드 검증기는 기본적으로 after 모드이다. 수업의 username 검증기는 문자열 검증과 최소 길이 검사 이후 strip().lower()를 수행한다. 이 순서에는 중요한 경계 사례가 있다. 앞뒤에 공백 두 개씩 붙인 입력 "A"는 전체 길이 5로 최소 길이 3을 통과하지만, 정규화 결과는 길이 1인 "a"가 된다. 해당 결과를 추가 입력으로 실제 확인하였다. [L3; R3]

5.2 정규화 뒤에도 제약이 유지되어야 하는 이유

최종 아이디의 길이를 제한하려면 공백 제거를 길이 검사보다 먼저 수행하거나, 정규화한 뒤 길이를 다시 확인해야 한다. 다음은 수업 원본을 설명하기 위해 구성한 개선 예시이며, 원본 파일을 수정한 결과는 아니다.

```
@field_validator("username", mode="before")
@classmethod
def normalize_username(cls, value: object) -> object:
    if isinstance(value, str):
        text = value.strip().lower()
        if " " in text:
            raise ValueError("username contains spaces")
        return text
    return value
# username: str = Field(min_length=3) is checked afterward.
```

5.3 필드 간 관계 검증

model_validator(mode="after")는 각 필드가 검증된 모델에서 관계를 검사한다. Signup은 password와 password_confirm이 같아야 하며, Reservation은 start가 end보다 늦을 수 없다. 조건이 어긋나면 ValueError를 발생시키고, 유효하면 self를 반환한다. 이 모델은 날짜가 같은 경우를 허용한다. [L3]

Reservation의 원본 오류 메시지는 조건과 반대로 적혀 있다. 실제 if self.start > self.end가 거부하는 상황은 시작일이 종료일보다 늦은 경우이다. 따라서 메시지는 "종료일은 시작일보다 빠를 수 없습니다"가 정확하다. 오류 메시지는 검증 조건과 동일한 규칙을 설명해야 한다.

6 중첩 모델과 복합 데이터 검증

6.1 객체의 관계를 모델로 표현하기

주문은 고객명만으로 표현되지 않는다. 배송 주소와 여러 주문 항목이 함께 필요하다. Order는 Address와 list[OrderItem]을 필드로 두어 이러한 관계를 표현한다. raw 딕셔너리를 Order.model_validate에 전달하면 내부 address는 Address 객체로, items의 각 딕셔너리는 OrderItem 객체로 검증된다. 따라서 order.address.city와 같은 속성 접근이 가능해진다. [L4]

```
class Address(BaseModel):
    city: str
    street: str

class OrderItem(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(ge=1)
    unit_price: int = Field(ge=0)

class Order(BaseModel):
    customer_name: str
    address: Address
    items: list[OrderItem] = Field(min_length=1)
```

이 구조는 목록 자체의 조건과 개별 항목의 조건을 분리한다. 주문 항목 목록은 비어 있으면 안 되고, 항목이 존재해도 quantity가 0이면 안 된다. 한 번의 최상위 모델 검증에서 이러한 내부 규칙을 함께 검사한다. 중첩 딕셔너리를 단순히 저장하는 방식보다 오류를 데이터 구조와 연결하기 쉽다. [L4]

6.2 Enum과 고정된 선택지

DeliveryMethod(str, Enum)는 STANDARD="standard", EXPRESS="express"를 정의한다. Enum의 name은 STANDARD와 같은 멤버 이름이고, value는 "standard"와 같은 외부 값이다. 문자열 입력 "express"는 모델에서 해당 Enum 멤버로 해석된다. Literal은 간단한 허용값 집합에 적합하고, Enum은 선택지에 이름을 부여하여 여러 위치에서 재사용할 때 의미가 분명하다. [L2; L4]

6.3 파생 값과 오류 경로

Order.total은 각 항목의 quantity × unit_price 합계이다. Course.average_score는 점수 합계를 수강생 수로 나눈 값이며, 수업 입력 50점과 70점의 평균은 60.0이다. learners의 min_length=1이 유효한 생성 시점에 빈 목록을 거부하므로 이 조건 아래에서 평균의 분모는 0이 아니다. [L4]

@property는 total() 대신 total로 값을 읽게 하지만, 일반 property가 자동으로 model_dump 결과에 포함되는 것은 아니다. 또한 ValidationError.errors()의 loc는 내부 오류 경로를 나타낸다. 첫 수강생의 점수가 101이면 ("learners", 0, "score"), 첫 주문 수량이 0이면 ("items", 0, "quantity")를 통해 문제 위치를 찾을 수 있다. [L4]

7 직렬화와 데이터 표현의 구분

7.1 검증과 직렬화의 방향

검증은 외부 입력을 규칙에 맞는 모델 객체로 만드는 과정이고, 직렬화는 모델을 딕셔너리나 JSON처럼 전달 가능한 표현으로 바꾸는 과정이다. 같은 모델이라도 출력 방식에 따라 포함되는 값과 타입이 다를 수 있다. 따라서 "딕셔너리로 만들었다"와 "JSON으로 전송할 수 있다"를 동일하게 취급하지 않아야 한다. [L5; R5]

메서드	입력 또는 결과	역할
Task.model_validate(raw)	딕셔너리 → Task	타입과 제약을 검사하여 객체 생성
task.model_dump()	Task → dict	기본 Python 표현으로 출력
task.model_dump(mode="json")	Task → dict	JSON에 맞는 값 타입으로 출력
task.model_dump_json()	Task → str	JSON 문자열로 출력

Task의 due는 date | None이다. due="2026-11-01"로 생성하면 모델 안에서는 날짜 객체가 되고, 기본 model_dump 결과에도 날짜 객체가 남을 수 있다. model_dump_json은 날짜를 JSON 문자열로 표현한다. 코드에서 Python의 None은 JSON의 null, True와 False는 JSON의 true와 false로 표현된다. [L5]

7.2 생략한 필드와 명시한 기본값

```
class ProfilePatch(BaseModel):
    nickname: str | None = None
    bio: str | None = None
    active: bool | None = None

ProfilePatch().model_dump(exclude_unset=True)
# {}
ProfilePatch(bio=None).model_dump(exclude_unset=True)
# {"bio": None}
```

두 객체 모두 bio를 읽으면 None이지만 입력 의도는 다르다. 첫 번째는 소개 글을 변경하지 않겠다는 요청이고, 두 번째는 소개 글을 비우겠다는 요청으로 사용할 수 있다. exclude_unset=True는 입력되지 않은 필드를 제외하므로 이 차이를 보존한다. exclude_none=True는 값이 None인 필드를 제외하므로 명시적인 비우기 요청도 사라질 수 있다. [L5]

7.3 기본값 생성과 객체의 독립성

Signup의 tags는 Field(default_factory=list)로 선언되어 있다. 이는 생략 시 호출할 생성 함수를 명시하는 방법으로, 각 객체의 목록이 독립적이라는 의도를 드러낸다. 반면 직렬화는 그 자체로 값을 다시 검증하거나 모델을 변경하지 않는다. 출력한 딕셔너리를 수정해 저장하려면 최종 모델 규칙으로 다시 확인하는 절차가 필요하다. [L3; L5]

8 부분 수정과 최종 상태의 재검증

8.1 PATCH의 데이터 흐름

부분 수정 요청에는 바꿀 필드만 들어온다. 수업의 TaskPatch는 각 필드의 기본값을 None으로 두어 생략을 허용한다. 이 객체를 그대로 전체 출력해 덮어쓰면, 생략된 필드의 기본값이 기존 값을 지울 수 있다. 따라서 입력 여부를 보존한 변경 딕셔너리를 만들고, 기존 상태와 병합한 뒤 최종 모델로 재검증한다. [L5; L6; R6]

```
changes = patch.model_dump(exclude_unset=True)
merged = {**stored.model_dump(), **changes}
updated = Task.model_validate(merged)
# Save updated only after successful validation.
```

딕셔너리 병합에서 뒤에 위치한 changes가 같은 키의 값을 덮어쓴다. 변경값 추출, 병합, 재검증, 저장의 순서가 핵심이다. 요청용 모델의 필드가 nullable이더라도 저장 모델의 필드는 nullable이 아닐 수 있다. 따라서 요청 모델을 통과했다는 사실만으로 저장 가능한 최종 상태임을 보장할 수 없다.

8.2 생략과 null을 처리하는 정책

요청	의미	수업 예제의 처리
{}	모든 필드 유지	API에서 200이며 상태 유지
{"done": true}	done만 변경	기존 title을 유지하고 수정
{"title": null}	title을 비우도록 요청	API 병합 뒤 TaskOut 검증에서 422
{"due": null}	기한을 비우도록 요청	직렬화 장의 Task에서는 허용

직렬화 장과 API 장은 거부 시점이 다르다. 직렬화 장의 TaskPatch는 after 모델 검증기에서 title과 done의 명시적인 None을 거부한다. due는 최종 Task에서도 None을 허용하므로 비울 수 있다. API 장의 TaskPatch는 title과 done의 None을 우선 허용하고, 병합 후 TaskOut 검증에서 이를 거부한다. 같은 정책을 어느 단계에서 구현하는지 비교할 수 있다. [L5; L6]

8.3 PUT과의 차이

수업의 PUT은 기존 객체의 존재만 확인하고 TaskIn으로 새 TaskOut을 만든다. 따라서 done=True인 객체에 {"title": "replaced"}만 PUT하면 done은 기존 True가 유지되지 않고 TaskIn의 기본값 False로 바뀐다. 같은 필드를 PATCH로 생략하면 기존 값을 유지한다. "전체 교체"와 "일부 변경"은 필수 입력의 수보다 기존 상태를 어떻게 다루는지로 이해해야 한다. 이 차이는 API 실행에서도 확인하였다. [L6; R6]

이 수업의 병합 대상은 단순한 작업 모델이다. 중첩 딕셔너리를 깊이 병합하거나, 관계형 데이터베이스에서 여러 레코드를 원자적으로 수정하는 문제까지 해결한 구현은 아니다. 그러한 확장은 별도의 갱신 정책과 저장 계층 설계가 필요하다.

9 FastAPI에서 모델을 API 계약으로 사용하기

9.1 요청 모델과 응답 모델

app.py의 TaskIn은 클라이언트가 보낼 title과 done을 정의한다. TaskOut은 이를 상속하여 서버가 부여하는 id를 추가한다. TaskPatch는 변경 가능한 필드의 입력 형식을 정의한다. 이 분리는 생성 시 클라이언트가 id를 정하게 하지 않으면서, 응답에서는 식별자를 제공하게 한다. [L6]

라우트의 payload: TaskIn은 요청 본문을 모델로 검증하게 하고, task_id: int는 경로 값을 정수로 해석하게 한다. response_model=TaskOut은 응답의 구조를 선언한다. 수업 코드에서 이 모델들은 저장 객체와 반환값에도 사용된다. 잘못된 요청과 서버의 잘못된 반환값은 서로 다른 실패이므로 응답 검증 실패까지 모두 클라이언트 입력 오류라고 부르면 안 된다.

9.2 CRUD 경로와 상태 코드

HTTP 메서드	경로	동작	성공 응답
GET	/tasks	목록 조회	200
GET	/tasks/{task_id}	단건 조회	200
POST	/tasks	작업 생성	201
PUT	/tasks/{task_id}	기존 작업 교체	200
PATCH	/tasks/{task_id}	일부 필드 수정	200
DELETE	/tasks/{task_id}	작업 삭제	204와 빈 본문

get_existing은 없는 식별자에 HTTPException(404)을 발생시킨다. 요청 스키마가 잘못되면 FastAPI가 422 응답을 만든다. PATCH 병합 뒤 발생하는 ValidationError는 수업 코드가 직접 잡아서 loc, msg, type으로 구성된 422 상세 정보를 반환한다. 이 수동 오류의 loc에는 자동 요청 검증 오류의 "body" 접두사가 없을 수 있다. [L6; R7]

9.3 실행 구조와 저장의 한계

_db는 dict[int, TaskOut]이고 _next_id는 다음 식별자를 보관한다. create_task는 모듈 변수의 값을 다시 대입하므로 global _next_id를 선언한다. 단순히 함수 밖 변수를 읽기 위해서 항상 global이 필요한 것은 아니다. 서버를 재시작하면 메모리 데이터는 사라지며, 여러 프로세스가 같은 딕셔너리를 공유하는 저장 구조도 아니다. [L6]

```
# Working directory: ch07_FastAPI
python -m uvicorn app:app --reload
# Open http://127.0.0.1:8000/docs
```

명령의 app:app은 app.py 모듈 안의 app 객체를 뜻한다. /docs에서 요청 본문과 응답 모델을 확인하며 실습할 수 있다. 브라우저 주소창 이동은 보통 GET이지만 브라우저 애플리케이션도 POST 등 다른 메서드를 보낼 수 있다. "브라우저는 GET, 프로그램은 POST"라는 구분은 정확하지 않다. [L6]

10 실행 검증 결과

10.1 파일 실행과 경계 입력

Python 파일 14개의 직접 실행 중 13개는 종료 코드 0으로 끝났고, ch.03_Field/exercise.py는 NameError로 종료되었다. 이는 파일 실행 기준의 관찰이다. 특히 app.py를 직접 실행하는 것은 서버를 구동해 요청을 처리하는 검증과 다르다. 별도의 TestClient 확인을 통해 다음 요청 동작을 관찰하였다. [L1-L6]

확인 항목	입력 또는 조건	관찰 결과
목록 조회	GET /tasks	200, 초기 작업 1개
생성과 정규화	POST title=" new "	201, id=2, title="new"
엄격한 불리언	POST done="true"	422, bool_type
공백 제목	POST title=" "	422, string_too_short
추가 키	POST foo=1 포함	422, extra_forbidden
부분 변경	PATCH done=true	200, 기존 title 유지
명시적 null	PATCH title=null	422, string_type
빈 수정 요청	PATCH {}	200, 상태 유지
전체 교체	PUT title만 전달	200, done=False로 재설정
없는 작업	GET /tasks/999	404
경로 타입	GET /tasks/abc	422, int_parsing
삭제	DELETE /tasks/2	204, 응답 본문 없음
삭제 후 조회	GET /tasks/2	404

10.2 결과의 해석

모델의 제약은 API 입력까지 연결되었으며, PATCH의 최종 재검증은 요청 모델에서 허용한 null이 저장 모델에 들어가는 것을 막았다. 별도 경계 입력에서는 username의 공백 포함 입력이 "a"로 바뀌면서 최종 최소 길이가 유지되지 않는 사례를 확인했다. Product의 price=0과 앞에 문자가 붙은 SKU도 원본 제약 아래에서 허용되었다. 이러한 결과는 선언 자체뿐 아니라 처리 순서와 경계값을 함께 검토해야 함을 보여 준다.

exercise_app.py의 생성 요청은 정상 구현이 아니라 501과 TODO 1 상세 정보를 반환했다. 정상 종료한 파일이라고 해서 실습이 완성되었다고 해석할 수 없다. 본 확인은 순차 요청을 사용한 기능 검증으로, 네트워크 지연, 부하, 동시성, 인증, 데이터베이스 지속성은 측정하지 않았다.

11 논의와 결론

11.1 원본 코드에서 구분해야 할 사항

상품 실습의 실행 중단. Product 실습은 product1을 생성하지만 반복문에서 product.model_dump()를 호출한다. product가 정의되지 않아 NameError가 발생하고, 그 아래의 누락 필드 검증 코드는 실행되지 못한다. product1을 사용하도록 바꾸는 것과 별도로, 반복문 안에서 Product.model_validate(data)를 호출하고 각 ValidationError를 처리해야 잘못된 입력 실습이 실제 검증으로 이어진다. [L2]

검증 조건과 실습 데이터의 불일치. Product의 price는 ge=0이므로 0은 유효하다. stock에는 default=0이 있으므로 생략할 수 있다. 따라서 "price와 stock이 모두 필수"라는 주석은 현재 선언과 맞지 않는다. 변경 딕셔너리의 sku는 실제 필드명 SKUcode와 다르다. 기본 extra 정책 아래에서 잘못된 키가 무시되면 기존 SKUcode가 남아 의도한 실패를 확인하지 못할 수 있다. [L2]

정규화와 오류 설명. username의 최소 길이는 정규화 후 값에서도 만족되어야 한다. Reservation의 오류 메시지도 실제 비교 조건과 일치시켜야 한다. 검증에 실패했다는 사실뿐 아니라 무엇을 수정해야 하는지 정확히 알려 주어야 학습자와 API 사용자가 규칙을 이해할 수 있다. [L3]

미완성 학습 단계. Student 예제의 잘못된 나이 처리 과제, 직렬화 exercise.py의 변경값 추출과 병합 과제, exercise_app.py의 생성·단건 조회·수정 구현은 모두 완성된 동작과 구분해야 한다. 원본의 TODO 표시는 학습 목표를 설명하지만 실행 결과를 입증하지는 않는다. [L1; L5; L6]

11.2 데이터 규칙을 유지하는 설계 원리

수업의 개념은 하나의 처리 흐름으로 연결된다. BaseModel이 구조를 정하고, Field가 단일 값의 조건을 정한다. field_validator는 입력을 정규화하며, model_validator는 여러 필드의 관계를 검사한다. 중첩 모델은 이 규칙을 내부 구조로 확장한다. 직렬화는 데이터를 외부 표현으로 바꾸고, PATCH는 입력 여부를 보존하여 변경분만 추출한다. 최종 재검증을 거친 상태가 FastAPI 응답과 저장의 대상이 된다.

다만 생성 시 검증을 통과한 모델이 이후 모든 변경에서도 항상 유효하다고 볼 수는 없다. 원본 예제는 대입 검증이나 변경 불가능성까지 구성하지 않았다. 따라서 모델 속성이나 내부 리스트를 나중에 직접 바꾸는 코드에도 별도의 정책이 필요하다. 요청 검증은 입력 경계의 규칙이며, 업무 처리 과정 전체의 무결성을 자동으로 대체하지 않는다.

11.3 결론

Day6의 핵심은 타입 선언을 실행 가능한 데이터 규칙으로 연결하고, 그 규칙을 API의 생성과 수정 과정에 일관되게 적용하는 것이다. 특히 생략과 None 허용을 구분하고, 정규화 후 제약이 유지되는지 확인하며, 부분 수정 후 최종 상태를 재검증해야 한다. 다음 학습 단계는 남아 있는 TODO를 완성하고, 경계 입력을 재현한 뒤, 메모리 저장을 지속 가능한 저장 계층으로 확장하는 것이다. 본고의 기능 확인 결과는 이러한 학습 순서를 뒷받침하며 운영 수준의 성능이나 안전성을 일반화하는 근거로 사용하지 않는다.

참고문헌

수업 자료와 공식 문서를 구분하여 표기하였다. 공식 문서의 열람일은 2026년 10월 21일이다. URL은 클릭하여 원문을 열 수 있다.

수업 소스

[L1] Day6 수업 코드. ch.02_basemodel. day6_1.py, day6_2.py, day6_2_1.py, day6_2_1_ex.py, main.py. BaseModel, 필수 필드, 타입 표현과 모델 상속.

[L2] Day6 수업 코드. ch.03_Field. demo.py, exercise.py. Registration과 Product의 값 제약 및 실습.

[L3] Day6 수업 코드. ch.04_validator/demo.py. Signup과 Reservation의 사용자 정의 검증.

[L4] Day6 수업 코드. ch.05_nested. demo.py, exercise.py. 주문 및 강좌 데이터의 중첩 모델.

[L5] Day6 수업 코드. ch.06_serialization. demo.py, exercise.py. Task와 Profile의 직렬화 및 부분 수정.

[L6] Day6 수업 코드. ch07_FastAPI. app.py, exercise_app.py. 메모리 기반 작업 CRUD와 미완성 실습 API.

공통 소스 경로 C:/AIO4/Day6/06_Pydantic_데이터검증_FastAPI/

공식 기술 문서

[R1] Pydantic. Models.
https://docs.pydantic.dev/latest/concepts/models/

[R2] Pydantic. Fields.
https://docs.pydantic.dev/latest/concepts/fields/

[R3] Pydantic. Validators.
https://docs.pydantic.dev/latest/concepts/validators/

[R4] Pydantic. Strict Mode.
https://docs.pydantic.dev/latest/concepts/strict_mode/

[R5] Pydantic. Serialization.
https://docs.pydantic.dev/latest/concepts/serialization/

[R6] FastAPI. Body Updates.
https://fastapi.tiangolo.com/tutorial/body-updates/

[R7] FastAPI. Handling Errors.
https://fastapi.tiangolo.com/tutorial/handling-errors/